Autopsy Investigation Runbook

Linear disk investigation in Autopsy 4.x, aligned to the five questions

Run top to bottom. Each step says what to do, what to look for, and what to record. Autopsy 4.19+ labels the tree **Data Artifacts / Analysis Results**; older builds put everything under **Results → Extracted Content**. Path tables are in the Execution Reference §7B. Record in Hunt Notes with source `Autopsy`.

Where Autopsy fits. Same role as FTK: disk **proves and dates** what memory and live triage found (Q1–Q3) and answers Q4–Q5. Autopsy's advantages are that it decodes execution artifacts for you (Run Programs, Recent Activity), has a built-in **Timeline**, and shows `$SI` vs `$FN` times per file. Use it for Q5 even if FTK did the rest.

Step 0 — Pre-flight (5 min)

- ☐ Version: `Help → About`. Note it.
- ☐ Target host's **system time zone** known (live triage §1.1 / §2.1).
- ☐ Image hash verified (FTK Imager **Verify Drive/Image**, or the provided hash).
- ☐ Output drive/path chosen.
- ☐ Lead list from Hunt Notes ready: names, paths, IPs, hashes, accounts.

Step 1 — Create the case (3 min)

1. `Case → New Case` → name `CASE_<host>` → base directory on the output drive → Next → examiner info → Finish.
2. **Add Data Source** opens. Host: *Generate new host name based on data source name* → Next.
3. Type: **Disk Image or VM File** (E01, raw/dd, VMDK, VHD). Use **Local Disk** for a mounted image and **Logical Files** for a folder of exported artifacts.
4. Select the image → **time zone = the target's** → Next.

Step 2 — Ingest: fast set first (5 min setup, runs in background)

Run now	Add later if time (right-click data source → Run Ingest Modules)
Recent Activity	Keyword Search (full index)
File Type Identification	PhotoRec Carver
Hash Lookup (+ NSRL if loaded)	Plaso (full super-timeline, slow)
Extension Mismatch Detector	Embedded File Extractor
Interesting Files Identifier	Encryption Detection

Run now	Add later if time (right-click data source → Run Ingest Modules)
Data Source Integrity	YARA Analyzer (if present)

Load your lead list as keywords before running Keyword Search: `Tools → Options → Keyword Search → New List` → add IPs, filenames, `base64_decode` , `eval(` → run Keyword Search ingest on the data source.

Display in UTC: `Tools → Options → View` → time zone **GMT/UTC**. Then times you read go into Hunt Notes with zone `UTC` , no conversion.

Results populate as modules finish. **Start step 3 immediately.**

Step 3 — Fast triage (10 min)

#	Tree node	Look for	Feeds
a	Analysis Results → **Hashset Hits**	Known-bad hash matches	Q1
b	Analysis Results → **Extension Mismatch**	`.jpg` that is a PE	Q1
c	Analysis Results → **Interesting Items**	Rule matches (from Interesting Files Identifier)	Q1
d	File Views → File Types → **By Extension** → Executable, Scripts (`.ps1 .bat .vbs .php .sh`)	Sort Modified desc. User, Temp, ProgramData, Public, `/tmp` , `/var/www`	Q1, Q5
e	File Views → **Deleted Files**	Tools removed after use	Q1, Q5
f	File Views → **File Size**	Large archives (staging / exfil)	Q1

On every hit: right-click → **Add File Tag → Notable Item** → comment = why + Q. **Record** in Hunt Notes.

Step 4 — Q1: What is running that should not be? Prove it on disk (15 min)

For each process lead from memory/live triage:

1. **Data Sources** tree → navigate to the path. File present? Bottom pane → **File Metadata**: size, MD5/SHA-256, times.
2. **Hex** tab: `MZ` = PE, `7F 45 4C 46` = ELF.
3. Right-click → **View File in Directory**: neighbors dropped at the same time.
4. Right-click → **Extract File(s)** → hash on the analysis box.

Gate G1 → acquire and hash. Memory process + disk file = **two sources**, Confirmed for Q1.

Step 5 — Q3: How does it survive a reboot? (15 min)

- **Data Artifacts → Run Programs / Installed Programs / Operating System Information.** New software, odd service binaries.
- Data Sources → `Windows\System32\config\SOFTWARE` / `SYSTEM` / `Users\<u>\NTUSER.DAT` → select → **Application** tab shows the registry tree. Check Run/RunOnce, Services (`ImagePath` , `Parameters\ServiceDll`), Winlogon.
- `Windows\System32\Tasks\` → task XML → **Text** tab → `<Command>` .
- Linux: `/etc/cron*` , `/var/spool/cron/crontabs/` , `/etc/systemd/system/*.wants/` , `/home/*/.ssh/authorized_keys` , `/etc/ld.so.preload` , `/var/www/` .

Gate G3: persistence pointing at the Q1 file = **foothold confirmed**. **Record** entry, target, last-write time (Q3).

Step 6 — Q4: Who did it, and how did they get in? (15 min)

- **Data Artifacts → Operating System User Account**: accounts, created, last logon.
- **Data Artifacts → Web History / Web Downloads**: how a payload arrived (browser download, URL).
- **Data Artifacts → Recent Documents · Shell Bags · USB Device Attached**: user activity, removable media.
- Event logs: Data Sources → `Windows\System32\winevt\Logs\` → **Extract** Security/System/PowerShell/Sysmon `.evtx` → query on the analysis box (Execution Reference §1.7 queries with `-Path`).
- Linux: extract `/var/log/auth.log*` , `/var/log/apache2/access.log*` , `.bash_history` → Execution Reference §2.5 / §2.7 greps.

Gate G4: account + auth method = scope extends to other hosts. **Record** (Q4).

Step 7 — Q5: When did it start? (20 min)

This is where Autopsy beats FTK.

1. **Data Artifacts → Run Programs**: Prefetch-derived executions with run times and counts. Find the Q1 binary.
2. **Amcache / UserAssist / BAM**: Recent Activity parses these into Data Artifacts (Run Programs / Recent Activity nodes, depending on version). If not shown, extract `Amcache.hve` / `NTUSER.DAT` / `SYSTEM` and use the native reads in Execution Reference §1.8.
3. **Timestomp check:** select the file → **File Metadata** tab. It shows `$STANDARD_INFORMATION` and `$FILE_NAME` times. SI earlier than FN, or SI with zero sub-seconds = timestomped; trust FN and the execution artifacts.

4. **Timeline:** `Tools → Timeline` → zoom to the window around the earliest time from 1–3 → filter event types (file created/modified, program run, web). Read **what happened just before**: download, logon, web POST, archive extraction.

Gate G5: first-seen earlier than the alert = intrusion start. Two artifacts agreeing on first run = confirmed start. **Record** each anchor (Q5), citing the artifact.

Step 8 — Optional depth (only if time remains)

- Run **PhotoRec Carver** / **Plaso** ingest. Carved files have no path and no reliable time; corroborate.
- `$Recycle.Bin\<SID>\` → `$I` (original path + delete time) / `$R` (content).
- Analysis Results → **Encryption Detected** (packed / encrypted payloads).

Step 9 — Report (10 min)

Tags node → review every Notable Item. `Generate Report → HTML` (or Excel) → **Tagged results only**. Write each finding with the Execution Reference §9 report-out: finding, host, first/last seen UTC, evidence, **second independent source**, ATT&CK, mitigation.

Pitfalls

- Wrong time zone at Add Data Source. Every reported time is off by the offset.
- Waiting for ingest to finish. Browse triage nodes as they fill.
- Running every ingest module up front and losing an hour to carving/Plaso.
- Trusting `$SI` times without the File Metadata `$FN` check.
- Memory images: Autopsy's Volatility option is Vol2 and Windows-only. Do memory in Vol3 and add dumped files back as **Logical Files**.